

Financial Data Mining and Analysis

Chapter 11: Value at Risk

Chapter 11: VaR (Value at Risk)

Price of 100 shares

One sentence definition of VaR

An example

VaR based on the assumption of normality

A normal distribution

Standard normal distribution

VaR based on sorting

1st, 2nd, 3rd, and 4th moments

Modified VaR

VaR of a portfolio

Value of 100 shares of IBM stock

```
import yfinance as yf
ticker="ibm"
df=yf.download(ticker)
print(df.tail(1))
```

```
nStocks=100
wealth=nStocks*df.Close.iloc[-1]
wealth=round(wealth,2)
print(f"n={nStocks},wealth={wealth}")
```

```
In [171]: print(df.tail(1))
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2024-03-01	185.490005	188.380005	185.179993	188.199997	188.199997	4017300

```
In [172]: print(f"n={nStocks},wealth={wealth}")
n=100,wealth=18820.0

In [173]:
```

What is VaR (Value at Risk)?

The maximum loss with a confidence level over a predetermined period

Examples of VaR

- Today, I own 200 shares of DELL stocks worth \$2,942. The maximum loss tomorrow is \$239 with a 99% confidence level.
- Our mutual fund has a value of \$10m today. The maximum loss over the next 3 months is 0.5m with 95% confidence.
- The value of our bank is \$150m. The VaR of our bank is \$10m with 99% confidence over the next 6 months.

Normality assumption related to VaR

- The first method to calculate VaR is based on the normality assumption of stock returns.
- Thus, we should understand normal distributions.

Standard normal distribution

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

```
import scipy as sp  
sp.stats.norm.pdf(0)
```

```
from math import sqrt, pi  
1/sqrt(2*pi)
```

```
In [17]: from math import sqrt, pi
```

```
In [18]: 1/sqrt(2*pi)
```

```
Out[18]: 0.3989422804014327
```

```
In [19]:
```

Normal distributions

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

When $\mu=0$ and $\sigma=1$, we have a standard normal distribution

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

A formula to estimate VaR

$$VaR_{period} = position * (\mu_{period} - z * \sigma_{period})$$

VaR is the Value of Risk

Position is dollar amount of your portfolio

μ_{period} is the mean of the period

z depends on the confidence level

σ_{period} is the standard deviation of the return of your portfolio

For a normal distribution

- $z=2.33$ for a 99% confidence level
- $z=1.64$ for a 95% confidence level

A simplified formula

When the mean return is small, we drop it

$$VaR = p * z * \sigma$$

Try the above simplified version and compare the difference by using IBM daily data

A trivial issue: sign

- Sometimes, we got two values with different signs, see below. Usually, it has a negative value

$$VaR_{period} = position * (\mu_{period} - z * \sigma_{period})$$

- *When the mean return is small, we drop it, see the following formula. However, its value is positive*

- $VaR = p * z * \sigma$

z-value for a given significance level

```
from scipy.stats import norm  
alpha=0.05  
z=norm.ppf(alpha)  
print(z)
```

```
In [186]: print(z)  
-2.3263478740408408
```

```
In [187]:
```

Show the z-value for given confidence level

```
from scipy.stats import norm
confidence_level=0.99
z=norm.ppf(1-confidence_level)
print(z)
```

```
In [186]: print(z)
-2.3263478740408408
```

```
In [187]:
```

10-day VaR

- Two methods to calculate an n-day VaR
 - Method #1: calculate the mean and standard deviation of daily returns. Then apply the following formulae to calculate n-day mean and returns.
 - Method #2: calculate n-day returns first, then calculate their mean and standard deviation

Formulae for an n-day VaR

$$\left\{ \begin{array}{l} \mu_{n_day} = (\mu_{daily} + 1)^n - 1 \\ \sigma_{n_day} = \sigma_{daily} \sqrt{n} \\ VaR_{n_day} = p * [\mu_{n_day} + z \sigma_{n_day}] \end{array} \right.$$

$$\left\{ \begin{array}{l} \sigma_{n_day}^2 = n * \sigma_{daily}^2 \\ \sigma_{n_day} = \sqrt{n} * \sigma_{daily} \end{array} \right.$$

For the confidence levels of 99% and 95%, we have:

$$\begin{cases} VaR_{n_day} = p * [(\mu_{daily} + 1)^n - 1 - 2.33\sigma_{daily}\sqrt{n}] \\ VaR_{n_day} = p * [(\mu_{daily} + 1)^n - 1 - 1.64\sigma_{daily}\sqrt{n}] \end{cases}$$

Modified VaR (mVaR)

$$\left\{ \begin{array}{l} z = qnorm(1 - confident) \\ S = skewness(ret) \\ K = excess\ kurtosis(ret) \\ t = z + \frac{1}{6}(z^2 - 1)S + \frac{1}{24}(z^3 - 3z)K - \frac{1}{36}(2z^3 - 5z)S^2 \\ mVaR = position * (\mu - t * \sigma) \end{array} \right.$$

Unlike traditional VaR, which assumes normally distributed returns, mVaR accounts for "fat tails" and asymmetry (skewness and kurtosis), providing a more realistic risk assessment for assets like hedge funds.

Issues with the normality assumptions

- Many argue that stock returns do not necessarily follow a normal distribution
- The fat tail (on the left) will make our VaR underestimate the true risk
- Other moments do exist

stats.shapiro test (Shapiro-Wilk test)

- **Hypothesis**

- **Null Hypothesis:** The data is normally distributed.
- **Alternative Hypothesis:** The data is *not* normally distributed.

- **Interpreting Results**

- The test outputs a statistic and a p-value.
 - **$p > 0.05$:** Fail to reject the null hypothesis. The data is likely drawn from a normal distribution.
 - **$p \leq 0.05$:** Reject the null hypothesis. There is a statistically significant deviation from normality.

stats.anderson test (**Anderson-Darling test**)

- **Hypothesis**

- **Null Hypothesis:** The data comes from the specified distribution.
- **Alternative Hypothesis:** The data does not come from the specified distribution.

- **Interpreting Results**

- The test outputs a statistic and a p-value.
 - **$p > 0.05$:** Fail to reject the null hypothesis. The data is likely drawn from a specified distribution.
 - **$p \leq 0.05$:** Reject the null hypothesis. There is a statistically significant deviation from a specified distribution.

VaR based on sorted historical returns

$$\left\{ \begin{array}{l} n = \text{length}(\text{ret}) \\ t = \text{as.integer}((1 - \text{confident}) * n) \\ \text{VaR} = \text{position} * \text{sorted_ret}[t] \end{array} \right.$$

Step 1: Sort returns from the lowest to the highest

Step 2: Choose appropriate return from the lowest ret as
our cut-off point

Step 3: VaR = position times * this return

VaR based on sorted historical returns

- **Historical Value at Risk (VaR)** using the historical simulation method (sorting historical returns to find the worst-case scenario at a specific confidence level).

Skewness and kurtosis

- For a normal distribution, its skewness is zero and kurtosis is 3.

$$skewness = \frac{\sum_{i=1}^n (R_i - \bar{R})^3}{(n-1)\sigma^3}$$

$$kurtosis = \frac{\sum_{i=1}^n (R_i - \bar{R})^4}{(n-1)\sigma^4}$$

- Sometimes, kurtosis is defined as below.

$$kurtosis = \frac{\sum_{i=1}^n (R_i - \bar{R})^4}{(n-1)\sigma^4} - 3$$

Formulae

$$\left\{ \begin{array}{l} \textit{skewness}(x) = \frac{\sum_{i=1}^n (x_i - \bar{x})^3}{(n-1)\sigma^3} \\ \textit{kurtosis}(x) = \frac{\sum_{i=1}^n (x_i - \bar{x})^4}{(n-1)\sigma^4} \\ \textit{Excess kurtosis} = \frac{\sum_{i=1}^n (x_i - \bar{x})^4}{(n-1)\sigma^4} - 3 \end{array} \right.$$

Calculating skewness and kurtosis ourselves

```
import numpy as np
from scipy import stats
from numpy import random
#
np.random.seed(12345)
n=5000000
ret = random.normal(0,1,n)
mean=ret.mean()
std=ret.std()
```

```
skew=0
kurt=0
for r in ret:
    skew+=(r-mean)**3
    kurt+=(r-mean)**4
#
skew/=(std**3*(n-1))
kurt/=(std**5*(n-1))
print(f"mean ={round(mean,5)}")
print(f"std ={round(std,5)}")
print(f"skewness={round(skew,8)}")
print(f"kurtosis={round(kurt,8)}")
```

```
In [18]: print(f"skewness={round(skew,8)}")
skewness=-0.00040546

In [19]: print(f"kurtosis={round(kurt,8)}")
kurtosis=2.99933517

In [20]:
```

Generate 4 moments from a standard normal

```
import numpy as np
from scipy import stats
from numpy import random
```

```
np.random.seed(12345)
n=5000000
#
ret = random.normal(0,1,n)
skew=round(stats.skew(ret),5)
kurt=round(stats.kurtosis(ret),5)
print(f"mean ={round(ret.mean(),5)}")
print(f"std ={round(ret.std(),5)}")
print(f"skewness={skew}")
print(f"kurtosis={kurt}")
```

- For a standard normal distribution
- Mean=0, std=1
- Skew=0, Kurt =0 or 3

```
In [184]: print(f"mean ={round(ret.mean(),5)}")
...: print(f"std ={round(ret.std(),5)}")
...: print(f"skewness={skew}")
...: print(f"kurtosis={kurt}")
mean =0.00036
std =0.99983
skewness=-0.00041
kurtosis=-0.00116

In [185]:
```

Based on the previous slide, we know the definition of stats.kurtosis is based on minus 3.

Expected shortfall

$$ES = (loss|z < -\alpha) = \frac{\int_{-\infty}^{-\alpha} x f(x) dx}{\int_{-\infty}^{-\alpha} f(x) dx} = \frac{-\phi(\alpha)}{F(\alpha)}$$

where ES is the expected shortfall

- Using R codes

- $ES = (loss|z < -\alpha) = \frac{-dnorm(qnorm(1-divident),0,sigma)}{1-confident}$

VaR of a 2-stock portfolio

- *For a two-stock portfolio*

$$\left\{ \begin{array}{l} \sigma^2(R_p) = x_1^2 \sigma_1^2 + x_2^2 \sigma_2^2 + x_1 x_2 \text{cov}(R_1, R_2) \\ \sigma^2(R_p) = x_1^2 \sigma_1^2 + x_2^2 \sigma_2^2 + x_1 x_2 \rho_{1,2} \sigma_1 \sigma_2 \end{array} \right.$$

Volatility of a 2-stock portfolio

```
# volatility of two-stock portfolio
import numpy as np
#
def portfolio_vol_2stocks(ret1,ret2,w1):
    x1=w1
    x2=1-x1
    s1=ret1.std()
    s2=ret2.std()
    v=x1**2*s1**2+x2**2*s2**2+2*x1*x2*np.cov(ret1,ret2)
    return(sqrt(v))
```

Variance of an n-stock portfolio

- $\sigma^2(R_p) = \sum_{i=1}^n \sum_{j=1}^n w_i w_j \sigma_{i,j}^2$

$\sigma^2(R_p)$ is the variance of an n-stock portfolio.

w_i is weight of stock i.

$\sigma_{i,j}^2$ is the variance between stocks i and j.

VaR for portfolios

One basic concept is:

VaR for a portfolio is not the weighted VaR of individual stocks

VaR for portfolios

```
import numpy as np
from math import sqrt
def vol_portfolio(ret_matrix,weights):
    m=ret_matrix.shape[1]
    m2=len(weights)
    if(m!=m2):
        print("# of stocks not = # of weights!")
    else:
        cov=np.cov(ret_matrix.T)
        final=0
        n=np.arange(0,m)
        for i in n:
            for j in n:
                final+=weights[i]*weights[j]*cov[i,j]
    stdP=sqrt(final)
    return(stdP)
```

Financial Data Mining and Analysis

Chapter 12: Monte Carlo Simulations

Agenda

- A simple example
- `np.random`
- Uniform distribution
- Standard normal distribution
- Normal distribution
- Correlation
- Correlated random numbers

Why are normal distributions so important for finance?

- There are the most important reasons:
 - 1) if our model is good, the error terms should follow a normal distribution with a mean error of zero.
 - 2) In finance, we assume that stock prices follow a log-normal distribution, stock returns follow a normal distribution, and stock prices follow a log-normal distribution.

numpy.random

```
import numpy.random as random
print(dir(random))
```

```
print(dir(random))
['BitGenerator', 'Generator', 'MT19937', 'PCG64',
 'PCG64DXSM', 'Philox', 'RandomState', 'SFC64',
 'SeedSequence', '__RandomState_ctor', '__all__',
 '__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__path__',
 '__spec__', '_bounded_integers', '_common',
 '_generator', '_mt19937', '_pcg64', '_philox',
 '_pickle', '_sfc64', '_beta', '_binomial',
 'bit_generator', 'bytes', 'chisquare', 'choice',
 'default_rng', 'dirichlet', 'exponential', 'f',
 'gamma', 'geometric', 'get_bit_generator',
 'get_state', 'gumbel', 'hypergeometric', 'laplace',
 'logistic', 'lognormal', 'logseries', 'mtrand',
 'multinomial', 'multivariate_normal',
 'negative_binomial', 'noncentral_chisquare',
 'noncentral_f', 'normal', 'pareto', 'permutation',
 'poisson', 'power', 'rand', 'randint', 'randn',
 'random', 'random_integers', 'random_sample',
 'ranf', 'rayleigh', 'sample', 'seed',
 'set_bit_generator', 'set_state', 'shuffle',
 'standard_cauchy', 'standard_exponential',
 'standard_gamma', 'standard_normal', 'standard_t',
 'test', 'triangular', 'uniform', 'vonmises', 'wald',
 'weibull', 'zipf']
```

A simple example

```
import numpy as np
```

```
x=np.random.standard_normal(size=10)
```

```
print(x)
```

```
In [217]: print(x)
[ 1.59732458 -0.32070043 -0.81966822 -0.21179606 -0.51997238  1.88394781
 -0.40312476 -0.98301365 -1.17011862 -1.52835189]
```

```
In [218]:
```

Concept of a random seed

We must choose the same random seed for a given distribution to get the same set of random values.

```
import numpy.random as random
random.seed(12345)
x=random.standard_normal(size=10)
print(x)
```

```
In [245]: print(x)
[-0.20470766  0.47894334 -0.51943872 -0.5557303  1.96578057  1.39340583
 0.09290788  0.28174615  0.76902257  1.24643474]
```

comment out the second line to see the effect

Random values from a uniform distribution

```
import numpy.random as random
random.seed(123345)
x=random.uniform(low=1,high=100,size=10)
print(x[0:5])
```

```
In [249]: print(x[0:5])
[30.32749021 20.58006409  2.43703988 76.15661293 75.06929084]

In [250]:
```


Randomly choose 15 values from [1,100]

```
import numpy as np
import numpy.random as random
#
random.seed(123345)
x=random.uniform(low=1,high=100,size=15)
print(x)
```

```
[30.32749021 20.58006409  2.43703988 76.15661293 75.06929084 25.59334724
 60.70199745 31.74285415 55.46365043 72.40727558 35.03271759 45.82660358
 12.9420188  67.14203485 90.01386334]
y values =[30 20  2 76 75 25 60 31 55 72 35 45 12 67 90]
The length of y= 15
The length of y2= 15

In [262]:
```

Return matrix from a uniform distribution

```
import numpy.random as random
random.seed(1234567)
nReturns=100
nStocks=5
minRet=-0.1
maxRet=0.05
returns=
random.uniform(minRet,maxRet,size=[nReturns,nStocks])
print(returns[0:2,])
```

```
In [385]: print(returns[0:2,])
[[-0.06444562 -0.09885274 -0.09702545 -0.05303611 -0.0850818 ]
 [-0.07072386 -0.0689053  -0.07526032  0.00678184 -0.09519   ]]

In [386]:
```

Random values from a standard normal distribution

```
import numpy.random as random  
random.seed(12345)  
x=random.normal(0,1,20)  
print(x)
```

```
[-0.20470766  0.47894334 -0.51943872 -0.5557303  1.96578057  1.39340583  
 0.09290788  0.28174615  0.76902257  1.24643474  1.00718936 -1.29622111  
 0.27499163  0.22891288  1.35291684  0.88642934 -2.00163731 -0.37184254  
 1.66902531 -0.43856974]
```

Roll a dice function

```
import random
def rollDice():
    roll = random.randint(1,6)
    return roll
i =1
n=10
result=[]
#random.seed(123)
while i<n:
    result.append(rollDice())
    i+=1
print(result)
```

With/without replacements

Assume that we have the historical data, such as price and return, for a stock. We could estimate their mean, standard deviation, and other related statistics.

What are their expected annual mean and risk next year?

- The simplest, maybe naïve way is to use the historical mean and standard deviation.
- A better way is to construct the distribution of annual return and risk. This means we must find a way to use historical data more effectively to predict the future.
- In such cases, we could apply the bootstrapping methodology. For example, for one stock, we have its last 20-year monthly returns, that is, 240 observations

Random.permutation() function

```
import numpy as np
x=np.arange(1,11)
print(f"raw data set={x}")
#
for i in np.arange(1,6):
    y=np.random.permutation(x)
    print(f"resampling i={i}, new
data={y}")
```

```
In [478]: print(f"raw data set={x}")
raw data set=[ 1  2  3  4  5  6  7  8  9 10]

In [479]: for i in np.arange(1,6):
...:     y=np.random.permutation(x)
...:     print(f"resampling i={i}, new data={y}")
resampling i=1, new data=[ 9  2  4  3 10  7  6  5  8  1]
resampling i=2, new data=[ 9  2  1  4  6  5  7 10  8  3]
resampling i=3, new data=[10  6  4  7  9  3  1  2  5  8]
resampling i=4, new data=[ 5  9  2  6  3  4  1  8 10  7]
resampling i=5, new data=[ 7  6  4  9  1  8  5 10  2  3]
```

Simulation of stock terminal price movements

$$S_{t+1} = S_t + \hat{\mu}S_t\Delta t + \sigma S_t\epsilon\sqrt{\Delta t}$$

$$S_{t+1} = S_t \exp((\hat{\mu} - \frac{1}{2}\sigma^2)\Delta t + \sigma\epsilon\sqrt{\Delta t})$$

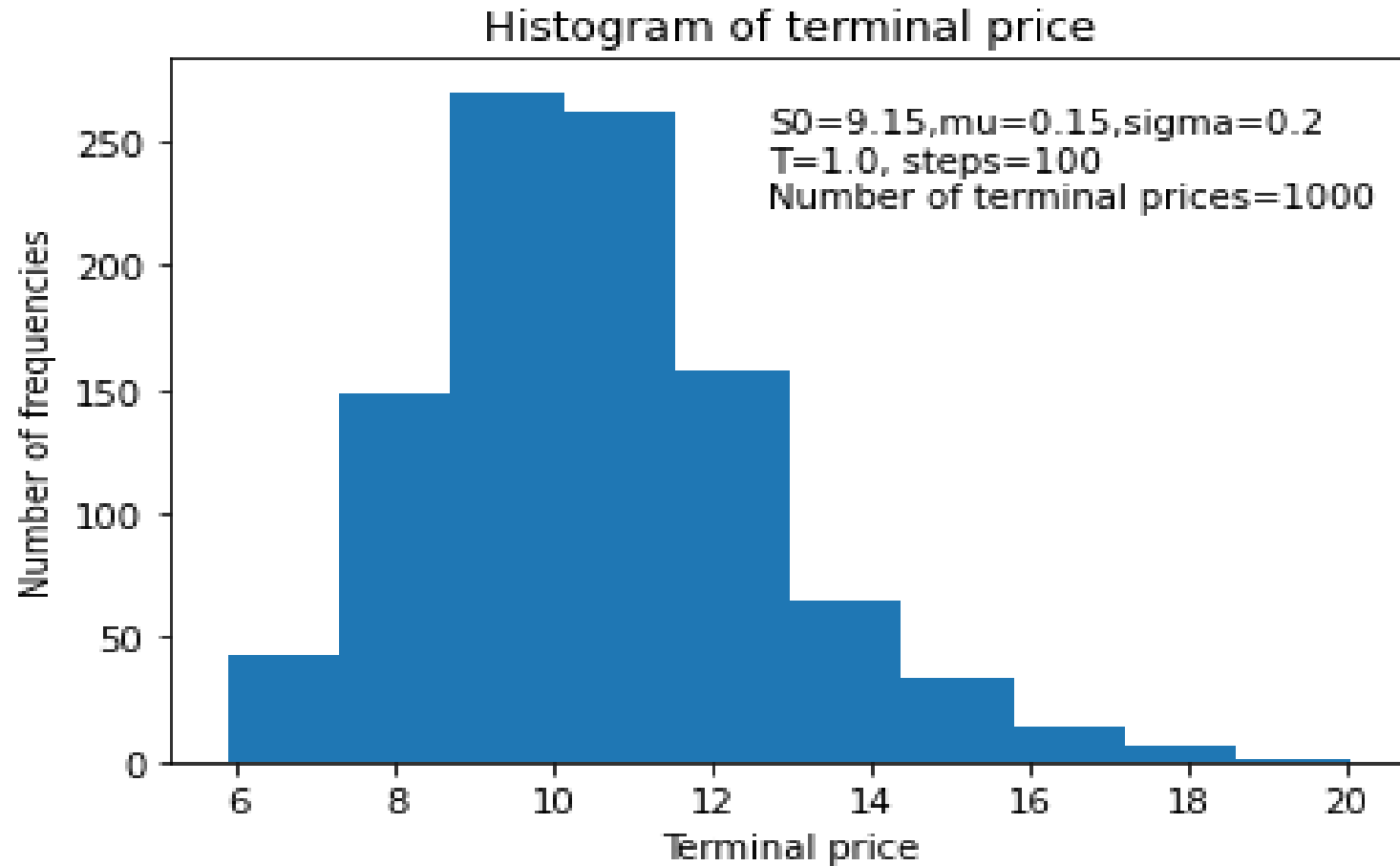
$$S_{t+1} = S_t \exp((r - \frac{1}{2}\sigma^2)\Delta t + \sigma\epsilon\sqrt{\Delta t})$$

Simulation of stock terminal price movements

```
import scipy as sp
from math import sqrt
import numpy.random as random
import matplotlib.pyplot as plt
#input area
S0 = 9.15                # stock price at time zero
T =1.                    # years
n_steps=100              # number of steps
mu =0.15                 # expected annual return
sigma = 0.2              # volatility (annual)
random.seed(12345)       # fix those random numbers
n_simulation = 1000      # number of simulation
dt =T/n_steps
#
S = np.zeros([n_simulation], dtype=float)
x = np.arange(0, int(n_steps), 1)
```

```
for j in np.arange(0, n_simulation):
    tt=S0
    for i in x[:-1]:
        e=np.random.normal()
        tt+=tt*(mu-0.5*pow(sigma,2))*dt+sigma*tt*sqrt(dt)*e;
    S[j]=tt
#
plt.title('Histogram of terminal price')
plt.ylabel('Number of frequencies')
plt.xlabel('Terminal price')
plt.figtext(0.5,0.8,'S0='+str(S0)+' ,mu='+str(mu)+' ,sigma='+str(sigma))
plt.figtext(0.5,0.76,'T='+str(T)+' ,
steps='+str(int(n_steps)))
plt.figtext(0.5,0.72,'Number of terminal
prices='+str(int(n_simulation)))
plt.hist(S)
plt.show()
```


Graph from the previous slide



Simulation and finance

- This method was first introduced to corporate finance by Hertz (1964).
- Later, Boyle (1977) uses simulation to price various options.

Black-Scholes-Merton by simulation (explained later)

```
for j in np.arange(0, n_simulation):
    ST=S0
    for i in x[:-1]:
        e=random.normal()
        ST*=exp((r-0.5*sigma*sigma)*dt+sigma*e*sqrt(dt))
    call[j]=max(ST-X,0)
#
call_price=round(call.mean()*exp(-r*T) ,4)
print(f"call price = {call_price}")
```

Path independent vs. path dependent

- Some options are path-independent, such as European call and put options.
 - The reason is that the option price on maturity depends on the underlying stock price and the exercise price. It has nothing to do with the path.
- Some options are path-dependent, such as Asian options (explained later).

Call simulation

```
import numpy as np
import numpy.random as random
from math import exp,sqrt
#
S0 = 40.      # stock price at time zero
X= 40.      # exercise price
T=0.5        # years
r=0.05       # risk-free rate
sigma = 0.2   # annualized volatility
n_steps=100   # number of steps
random.seed(12345) # fix those random numbers
n_simulation = 5000 # number of simulation
dt =T/n_steps
call = np.zeros([n_simulation], dtype=float)
```

```
x = np.arange(0, int(n_steps), 1)
for j in np.arange(0, n_simulation):
    sT=S0
    for i in x[:-1]:
        e=random.normal()
        a=sigma*e*sqrt(dt)
        sT*=exp((r-0.5*sigma*sigma)*dt+a)
        call[j]=max(sT-X,0)
#
call_price=round(call.mean()*exp(-r*T) ,4)
print(f"call price = {call_price}")
```

```
In [546]: print(f"call price = {call_price}")
call price = 2.7483
```

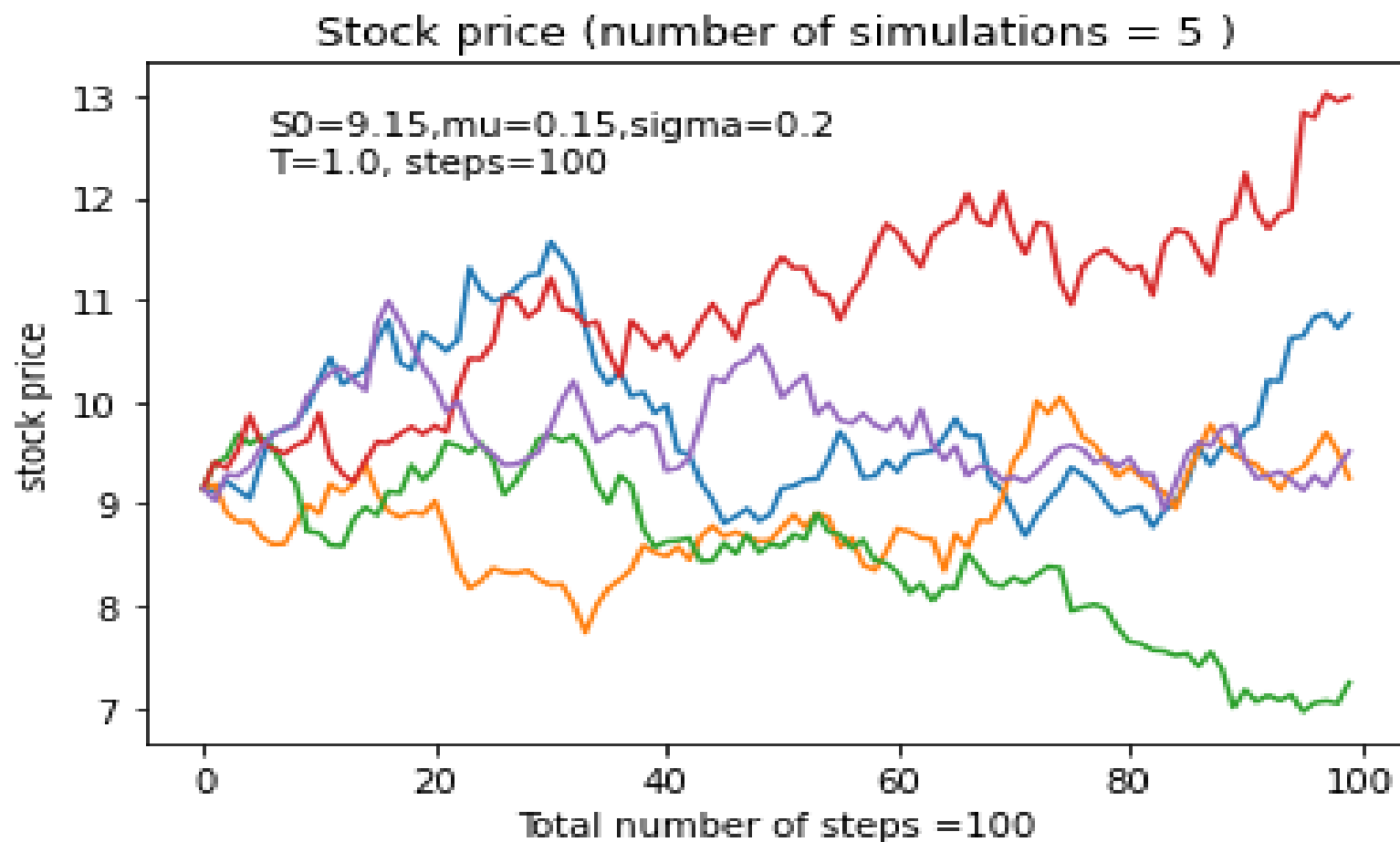
```
In [547]:
```

Paths of stock price

```
import numpy as np
from math import pow,sqrt
import numpy.random as random
import matplotlib.pyplot as plt
# input area
stock_price_today = 9.15 # stock price at time zero
T =1.                    # maturity date (in years)
n_steps=100              # number of steps
mu =0.15                 # expected annual return
sigma = 0.2              # annualized volatility
random.seed(12345)       # fixed our seed
n_simulation = 5         # number of simulations
dt =T/n_steps
S = np.zeros([n_steps],dtype=float)
x = range(0, int(n_steps), 1)
```

```
for j in range(0, n_simulation):
    S[0]= stock_price_today
    for i in x[:-1]:
        e=random.normal()
        aa=sigma*S[i]*sqrt(dt)*e
        S[i+1]=S[i]+S[i]*(mu-0.5*pow(sigma,2))*dt+aa;
    plt.plot(x, S)
#
plt.figtext(0.2,0.8,'S0='+str(S[0])+',mu='+str(mu)+' ,
sigma='+str(sigma))
plt.figtext(0.2,0.76,'T='+str(T)+' ,
steps='+str(int(n_steps)))
plt.title('stock price (number of simulations = %d '
% n_simulation +')')
plt.xlabel('Total number of steps
='+str(int(n_steps)))
plt.ylabel('stock price')
plt.show()
```

Visual presentation of 5 paths



Simulating stock prices (paths)

- The next stock price is a function of the current stock price, a tiny time interval, standard deviation, mean plus a random factor

$$S_{t+1} = S_t + \hat{\mu}S_t\Delta t + \sigma S_t\epsilon\sqrt{\Delta t},$$

In a risk-neutral world

- We have mean = risk-free rate
- The function of the next stock price is given below.

$$S_{t+1} = S_t \exp\left(r - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\epsilon\sqrt{\Delta t}$$

```
ST_f<-function(S,r,T,sigma){
```

```
  S*exp((r-0.5*sigma*sigma)*T+sigma*rnorm(1)*sqrt(T))
```

```
}
```

Price an Asian options

- European options are called path-independent options. It means that their final payoff has nothing to do with their path.
- Below is one example.

$$\text{payoff}(\text{Asian option, average price}) = \max(S_{\text{average}} - X, 0)$$

- depends on the path
 S_{average}

Why average instead of a terminal price?

- Assume that company A will pay 5 million pounds in three months. One option is to buy a call option with an exercise price. The company cares only about the exchange rate when it pays or exercises the option.
- One refinery uses crude oil every day. Although they worry about the price of crude oil, they are more concerned with the average price than with a specific day's price. The average price is better than the terminal price to hedge their risk.

Average price an Asian options

```
import numpy as np
import numpy.random as random
from math import exp,sqrt
#
s0=40.                # today stock price
x=40.                 # exercise price
T=0.5                 # maturity in years
r=0.05                # risk-free rate
sigma=0.2             # volatility (annualized)
random.seed(123)       # fix a seed here
n_simulation=100       # number of simulations
n_steps=100           # number of steps
dt=T/n_steps
call=np.zeros([n_simulation], dtype=float)
```

Correlated random sequences

Step 1: Generate two sequences of uncorrelated normal distributed random numbers, let's call them x_1 and x_2 .

Step 2: Generate y_1 and y_2 according to the following formulae.

$$y_1 = x_1$$

$$y_2 = \rho x_1 + \sqrt{1 - \rho^2} x_2$$

Correlated random series

```
import numpy as np
from math import sqrt
import numpy.random as random
#
random.seed(123)
n=1000
rho=0.3
x1=random.normal(size=n)
x2=random.normal(size=n)
#
y1=x1
y2=rho*x1+sqrt(1-rho**2)*x2
corrMatrix=np.corrcoef(y1,y2)
print(f"check new correlation matrix:\n = {corrMatrix}")
```

```
In [571]: print(f"check new correlation matrix:\n = {corrMatrix}")
check new correlation matrix:
= [[1.          0.28505213]
   [0.28505213 1.          ]]

In [572]:
```

Up-and-out call

- We have a normal call option.
- We add a fixed price. If the price of the underlying stock reach this price, then our option become zero. If the price never reaches this pre-fixed price, it is a normal option.

Up-and-out call

```
import numpy as np
import scipy.stats as stats
from math import log,exp,sqrt
import numpy.random as random
#
def bsCall(S,X,T,r,sigma):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    d2 = d1-sigma*sqrt(T)
    c=S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)
    return(c)
#
```

```
price of Black-Scholes call = 2.76
price (up-and-out-call)=0.781
```

```
In [647]:
```

```
def up_and_out_call(s0,x,T,r,sigma,n_simulation,barrier):
    n_steps=100
    dt=T/n_steps
    total=0
    for j in np.arange(0, n_simulation):
        sT=s0
        out=False
        for i in np.arange(0,int(n_steps)):
            e=random.normal()
            sT*=exp((r-0.5*sigma*sigma)*dt+sigma*e*sqrt(dt))
            if sT>barrier:
                out=True
        if out==False:
            total+=bsCall(s0,x,T,r,sigma)
    return(total/n_simulation)
#
s0=40.          # today stock price
x=40.           # exercise price
barrier=42      # barrier level
T=0.5           # maturity in years
r=0.05          # risk-free rate
sigma=0.2       # volatility (annualized)
n_simulation=1009 # number of simulations
#random.seed(12) # fix a seed
#
call=round(bsCall(s0,x,T,r,sigma),2)
result=up_and_out_call(s0,x,T,r,sigma,n_simulation,barrier)
print(f"price of Black-Scholes call = {call}")
print(f"price (up-and-out-call)={round(result,3)}")
```


Liking two methods for VaR using simulation

```
import numpy as np
import numpy.random as random
import scipy.stats as stats
#
position=1e6 # portfolio value
std=0.2 # volatility
mean=0.08 # mean return
confidence=0.99 # confidence level
nSimulations=50000 # number of simulations
#
# Method I
z=stats.norm.ppf(1-confidence)
VaR=round(position*(mean+z*std),2)
print(f"Holding={position}, VaR={VaR} tomorrow")
```

```
# Method II: Monte Carlo simulation
random.seed(12345)
ret2=random.normal(mean,std,nSimulations)
ret3=np.sort(ret2)
m=int(nSimulations*(1-confidence))
VaR2=round(position*(ret3[m]),2)
print(f"Holding= {position}, VaR2={VaR2} tomorrow")
```

```
Holding=1000000.0, VaR=-385269.57 tomorrow
Holding= 1000000.0, VaR2=-386113.46 tomorrow

In [595]:
```

Simulation for a capital budget (base case)

```
import scipy as sp
import numpy as np
import numpy_financial as npf
#
nYear=5                # number of years
costEquipment=5e6      # 5 million
n=nYear+1              # add year zero
price=28               # price of the product
units=100000           # estimate # of units sold
otherCost=100000       # other costs
sellingCost=1500       # selling/adm cost
R_and_D=200000         # Research and development
costRawMaterials=0.3   # % cost of raw materials
R=0.15                 # discount rate
tax=0.38               # corporate tax rate
```

```
sales=np.ones(n)*price*units
sales[0]=0              # sales for 1st year is zero
cost1=costRawMaterials*sales
cost2=np.ones(n)*otherCost
cost3=np.ones(n)*sellingCost
cost4=np.zeros(n)
cost4[0]=costEquipment
RD=np.zeros(n)
RD[0]=R_and_D          # assume R&D at time zero
D=np.ones(n)*costEquipment/nYear # straight line depreciation
D[0]=0                 # no depreciation at time 0
EBIT=sales-cost1-cost2-cost3-cost4-RD-D
NI=EBIT*(1-tax)
FCF=NI+D               # add back depreciation
npvProject=round(npf.npv(R,FCF),2) # estimate NPV
print("NPV of project=",round(npvProject,0))
```

```
NPV of project= 1849477.0
```

```
In [66]:
```

Arithmetic mean vs. geometric mean

Case #1: input values are not related to returns

Assume that we have n values: $x_1, x_2, \dots, x_n$

$$\text{arithmetic mean} = \frac{\sum_{i=1}^n x_i}{n}$$

$$\text{Geometric mean} = \left(\prod_{i=1}^n x_i \right)^{\frac{1}{n}}$$

For the geomean, assume we have three numbers, a, b, and c.

$$\text{Geomean} = (a*b*c)^{(1/3)}$$

a=2

b=4

c=5.3

$$(a*b*c)^{(1/3)}$$

```
In [240]: (a*b*c)**(1/3)
Out[240]: 3.487026802530256
In [241]:
```

Python code: Arithmetic/geometric means

```
import scipy.stats as stats
import numpy as np

print("\nCase #1: not related to returns")
x=[1,2,3,4,5]
a_mean=np.mean(x)
g_mean=round(stats.gmean(x),5)
print(f"input values are ={x}")
print(f"arimetic mean={a_mean}, geoMean={g_mean}\n")
```

```
Case #1: not related to returns
input values are =[1, 2, 3, 4, 5]
arimetic mean=3.0, geoMean=2.60517
```

```
In [226]:
```

Arithmetic and geometric mean for returns

Assume we have n returns: $R_1, R_2, \dots, R_n$

$$\text{arithmetic mean} = \frac{\sum_{i=1}^n R_i}{n}$$

$$\text{Geometric mean} = \left[\prod_{i=1}^n (R_i + 1) \right]^{\frac{1}{n}} - 1$$

For the geomean, assume we have three numbers, a, b, and c.

$$\text{Geomean} = [(a+1)*(b+1)*(c+1)]^{**}(1/3)-1$$

a=0.1

b=-0.04

c=0.03

$$((a+1)*(b+1)*(c+1))^{**}(1/3)-1$$

```
In [244]: ((a+1)*(b+1)*(c+1))**(1/3)-1
```

```
Out[244]: 0.028411791803753417
```

```
In [245]:
```

Arithmetic and geometric mean for returns

```
import scipy.stats as stats
import numpy as np
```

```
print("\nCase #2: for returns")
ret=[0.1,0.1,0.2,0,-0.05]
a_mean2=np.mean(ret)
retPlus1 = [x + 1 for x in ret]
g_mean2=round(stats.gmean(retPlus1)-1,5)
print(f"input returns are ={ret}")
print(f"arimetic mean={a_mean2}, geoMean={g_mean2}")
```

```
Case #2: for returns
input returns are =[0.1, 0.1, 0.2, 0, -0.05]
arimetic mean=0.07, geoMean=0.06644
```

```
In [228]:
```

Long-term forecast

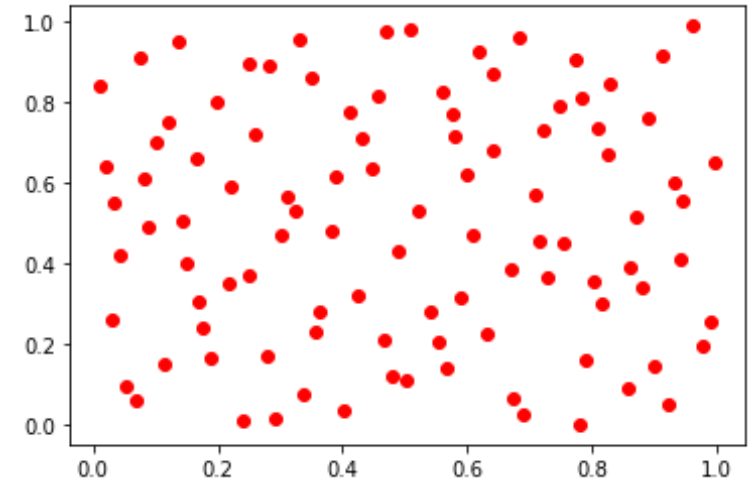
- For equity, using an arithmetic mean would overestimate short-term forecast returns
- On the other hand, using a geometric mean would underestimate short-term forecasted returns.

Sobol sequence

Sobol sequences are low-discrepancy, quasi-random numbers useful for various applications, including numerical integration, optimization, and Monte Carlo simulations.

Sobol sequence

```
import pandas as pd
import scipy.stats as stats
import matplotlib.pyplot as plt
d = 2    # Specify the dimensionality (e.g., 2, 3, etc.)
sobol_sequence = stats.qmc.Sobol(d, scramble=True)
n=100
sample = sobol_sequence.random(n) # Generate n points
df=pd.DataFrame(sample)
df.columns=["x", "y"]
plt.scatter(df.x, df.y, c="red")
plt.show()
```



Bootstrapping

- Bootstrapping is any test or metric that uses random sampling with replacement (e.g., mimicking the sampling process) and falls under the broader class of resampling methods.
- Bootstrapping assigns measures of accuracy (bias, variance, confidence intervals, prediction error, etc.) to sample estimates.
- This technique allows estimation of the sampling distribution of almost any statistic using random sampling methods.

Bootstrapping: boots_f() function

```
import numpy as np
def boots_f(data,n_obs,replacement=None):
    n=len(data)
    if (n<n_obs):
        print("n is less than n_obs")
    else:
        if replacement==None:
            y=np.random.permutation(data)
            return y[0:n_obs]
        else:
            y=[]
            for i in range(n_obs):
                k=np.random.permutation(data)
                y.append(k[0])
            return y
```

```
# test
x=[1,2,3,4,5,0,-9,2,-10]
print(f"x={x}")
y=boots_f(x,6)
print(f"u={y}")
```

```
In [10]: print(f"x={x}")
x=[1, 2, 3, 4, 5, 0, -9, 2, -10]

In [11]: y=boots_f(x,6)

In [12]: print(f"u={y}")
u=[ 5  2 -9  1  4  3]

In [13]: y=boots_f(x,6)

In [14]: print(f"u={y}")
u=[ 3  4  2 -9 -10  1]

In [15]:
```

Financial Data Mining and Analysis

Chapter 13: Exotic options

Agenda

- European, American, and Bermudan options
- Close-form solutions
- Path-independent vs. path-dependent options
- Simple chooser options
- Shout, rainbow, and binary options
- Average price option
- Barrier options
 - up-and-in options and up-and-out option
 - down-and-in and down-and-out options

What are exotic options?

- Exotic options are a class of derivative contracts that possess features or structures that differ from standard or vanilla options.
- These options are typically tailored to meet specific needs or to hedge against particular risks that may not be adequately addressed by standardized options contracts traded on exchanges.
- Exotic options often offer greater flexibility and customization but may also involve more complex payoff structures and pricing mechanisms.

Some common types of exotic options

1. Asian Options: The payoff of an Asian option depends on the average price of the underlying asset over a specified period rather than the asset's price at expiration.
2. Barrier Options: Barrier options come into existence or cease to exist if the price of the underlying asset reaches a predetermined barrier level during the life of the option. Examples include knock-in and knock-out options.
3. Binary Options: Binary options have a fixed payoff determined at the outset of the contract, depending on whether the underlying asset's price reaches a certain level or not.
4. Lookback Options: Lookback options provide the holder with the right to buy or sell the underlying asset at its highest or lowest price during the option's life.

Some common types of exotic options (2)

5. Compound Options: Compound options are options on other options. They provide the holder with the right to buy or sell another option at a predetermined price and date.

6. Chooser Options: Chooser options allow the holder to choose whether the option will be a call or put at a specified future date.

7. Bermudan Options: Bermudan options can be exercised at predetermined dates before expiration, similar to American options, but typically with limited exercise dates.

8. Quanto Options: Quanto options are denominated in one currency but settled in another, mitigating currency risk.

Exotic options

For example, if an option buyer could exercise their right several times before the maturity date, it is called a Bermudan option. Many exotic options (derivatives) may have several triggers relating to their payoffs. An exotic option may also include non-standard underlying security or instrument, developed for a specific client or for a particular market.

Exotic options are often traded over-the-counter (OTC) rather than on organized exchanges, and their prices may be determined through complex models or negotiated between counterparties. Due to their complexity and customization, exotic options may require sophisticated risk management techniques and pricing models for accurate valuation and analysis. Institutional investors, hedge funds, and corporations with specific risk management needs commonly use them.

Exotic: call or put options

```
from scipy import stats
from math import log,exp,sqrt
#
def call_or_Put(S,X,T,r,sigma,type='C'):
    d1=(log(S/X)+(r+sigma*sigma/2.)*T)/(sigma*sqrt(T))
    d2 = d1-sigma*sqrt(T)
    if type.upper()=='C':
        c=S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)
        return c
    else:
        p=X*exp(-r*T)*stats.norm.cdf(-d2)-S*stats.norm.cdf(-d1)
        return p
```

```
#
s=40
X=42
T=1
r=0.04
sigma=0.18
call_or_Put(s,X,T,r,sigma)
call_or_Put(s,X,T,r,sigma,"P")
```

```
In [72]: call_or_Put(s,X,T,r,sigma)
Out[72]: 2.7080216151418064

In [73]: call_or_Put(s,X,T,r,sigma,"P")
Out[73]: 3.061178059539383

In [74]:
```

Average price option

In Chapter 12, Monte Carlo Simulation, we discussed two exotic options. For convenience, we will include them in this chapter as well. Because of this, readers will find some duplicates.

European options are path-independent options. This means that an option's payoff depends only on the terminal stock price and strike price.

One related issue for path-dependent options is market manipulation at the maturity date. Another issue is that some investors or hedgers might care more about the average price instead of a terminal price

Barrier options

Unlike the Black-Scholes-Merton option model's call and put options, which are path-independent, a barrier option is path-dependent. A barrier option is similar in many ways to an ordinary option, except a trigger exists. An in option starts its life worthless unless the underlying stock reaches a predetermined knock-in barrier.

On the contrary, an out barrier option starts its life active and turns useless when a knock-out barrier price is breached. In addition, if a barrier option expires inactive, it may be worthless, or there may be a cash rebate paid out as a fraction of the premium.

The four types of barrier options are given in the next slide.

Four types of barrier-options

- Up-and-out: In this barrier option, the price starts from below a barrier level. If it reaches the barrier, it is knocked out.
- Down-and-out: In this barrier option, the price starts from above a barrier. If it reaches the barrier, it is knocked out.
- Up-and-in: In this barrier option, the price starts down a barrier and has to reach the barrier to be activated.
- Down-and-in: In this barrier option, the price starts over a barrier and has to reach the barrier to be activated

Using Monte Carlo Simulation to price barrier options

- For example, we have an up-and-call option. In this barrier option, the price starts from below a barrier level. If it reaches the barrier, the ordinal call is knocked out. The price of a given call is shown below.

`bs_call(s0,x,T,r,sigma)`

`bs_call(40,40,0.5,0.05,0.2) => 2.755`

For a given barrier such as 43, what is the up-and-out call?

We can run 200 simulations. Assume that 146 are not out. Then the price will be $(146/200) \times 2.755$

Simulation of stock terminal price movements

$$S_{t+1} = S_t + \hat{\mu}S_t\Delta t + \sigma S_t\epsilon\sqrt{\Delta t}$$

$$S_{t+1} = S_t \exp((\hat{\mu} - \frac{1}{2}\sigma^2)\Delta t + \sigma\epsilon\sqrt{\Delta t})$$

$$S_{t+1} = S_t \exp((r - \frac{1}{2}\sigma^2)\Delta t + \sigma\epsilon\sqrt{\Delta t})$$

Simulation of stock terminal price movements

```
import scipy as sp
from math import sqrt
import numpy.random as random
import matplotlib.pyplot as plt
#input area
S0 = 9.15                # stock price at time zero
T =1.                    # years
n_steps=100              # number of steps
mu =0.15                 # expected annual return
sigma = 0.2              # volatility (annual)
random.seed(12345)       # fix those random numbers
n_simulation = 1000      # number of simulation
dt =T/n_steps
#
S = np.zeros([n_simulation], dtype=float)
x = np.arange(0, int(n_steps), 1)
```

```
for j in np.arange(0, n_simulation):
    tt=S0
    for i in x[:-1]:
        e=random.normal()
        tt+=tt*(mu-0.5*pow(sigma,2))*dt+sigma*tt*sqrt(dt)*e;
    S[j]=tt
#
plt.title('Histogram of terminal price')
plt.ylabel('Number of frequencies')
plt.xlabel('Terminal price')
plt.figtext(0.5,0.8,'S0='+str(S0)+' ,mu='+str(mu)+' ,sigma='+str(sigma))
plt.figtext(0.5,0.76,'T='+str(T)+' ,
steps='+str(int(n_steps)))
plt.figtext(0.5,0.72,'Number of terminal
prices='+str(int(n_simulation)))
plt.hist(S)
plt.show()
```

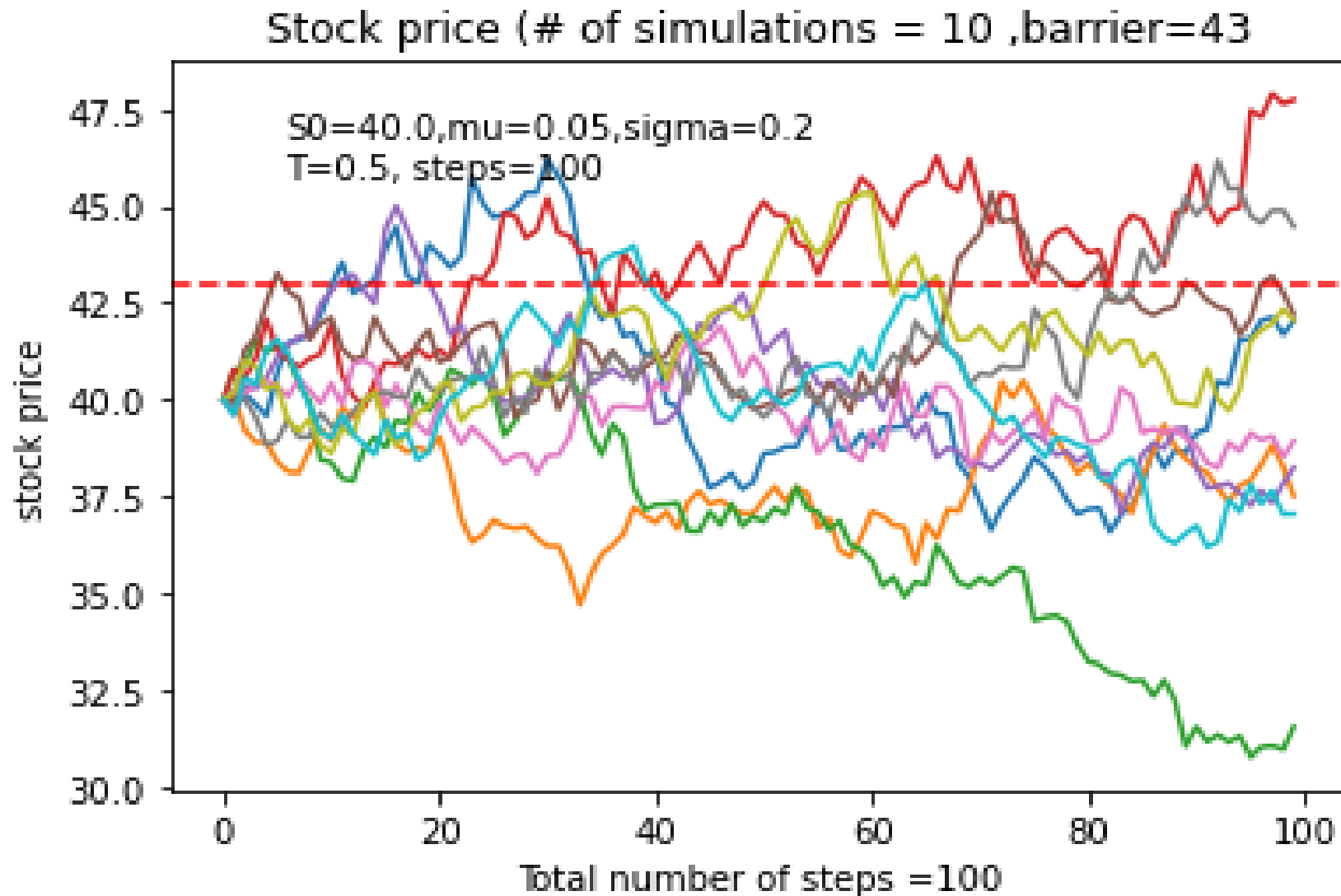

Paths of stock price with a barrier

```
import numpy as np
from math import pow,sqrt
import numpy.random as random
import matplotlib.pyplot as plt
# input area
barrier=43                # barrier
stock_price_today = 40    # stock price at time zero
T =0.5                    # maturity date (in years)
n_steps=100               # number of steps
mu =0.05                  # expected annual return
sigma = 0.2               # annualized volatility
random.seed(12345)        # fixed our seed
n_simulation = 10         # number of simulations
dt =T/n_steps
S = np.zeros([n_steps],dtype=float)
x = range(0, int(n_steps), 1)
```

```
for j in range(0, n_simulation):
    S[0]= stock_price_today
    for i in x[:-1]:
        e=random.normal()
        aa=sigma*S[i]*sqrt(dt)*e
        S[i+1]=S[i]+S[i]*(mu-0.5*pow(sigma,2))*dt+aa;
    plt.plot(x, S)

#
plt.figtext(0.2,0.8,'S0='+str(S[0])+',mu='+str(mu)+',sigma='+str(sigma))
plt.figtext(0.2,0.76,'T='+str(T)+', steps='+str(int(n_steps)))
aa='Stock price (# of simulations = %d ' % n_simulation +','
plt.title(aa+'barrier='+str(barrier))
plt.xlabel('Total number of steps =' +str(int(n_steps)))
plt.ylabel('stock price')
plt.axhline(y=barrier, color='red', linestyle='--')
plt.show()
```

Visual presentation stock path with a barrier



Chooser options

- A chooser option allows the option buyer to choose whether to purchase a European call or a European put at a predetermined point before the option matures.
- For a simple chooser option, the underlying call and put options have the same maturities and exercise prices.

What are the chooser options?

Chooser options, also known as "extendible options" or "delayed options," are a type of exotic option that grants the holder the right to choose the type of option (call or put) they wish to exercise at a predetermined future date.

This additional flexibility distinguishes chooser options from standard options, where the type of option (call or put) is fixed at the time of purchase.

The key features of chooser options

1. Flexibility: Chooser options allow the holder to assess market conditions and select the most advantageous option type (call or put) at a later date. This flexibility can be valuable in uncertain market environments or when the holder anticipates significant price movements but is unsure about the direction.
2. Exercise Period: The holder of a chooser option has the right to choose the option type (call or put) during a specified exercise period, which typically occurs before the option's expiration date. The exercise period may be a single date or a range of dates.
3. Premium: Chooser options generally command a higher premium compared to standard options due to the additional flexibility they offer. The premium is influenced by factors such as the underlying asset's volatility, time to expiration, and the length of the exercise period.

The key features of chooser options (2)

4. Expiration: Chooser options have a fixed expiration date, like standard options. The holder must exercise the option within the specified exercise period or let it expire worthless.
5. Risk Management: Chooser options can be used for risk management purposes, allowing the holder to adapt their option position based on evolving market conditions and their outlook for the underlying asset.
6. Complexity: Due to their added flexibility, chooser options are considered more complex than standard options. As a result, they are typically traded over-the-counter (OTC) rather than on organized exchanges and may require customized terms to meet the specific needs of the parties involved.

Chooser option: call or put

```
from scipy import stats
from math import log, exp, sqrt
def call_or_Put(S,X,T,r,sigma,tao,type='C'):
    d1=(log(S/X)+r*T+0.5*sigma*sigma*tao)/(sigma*sqrt(tao))
    d2 = d1-sigma*sqrt(tao)
    if type.upper()=='C':
        c=S*stats.norm.cdf(d1)-X*exp(-r*T)*stats.norm.cdf(d2)
        return c
    else:
        p=X*exp(-r*T)*stats.norm.cdf(-d2)-S*stats.norm.cdf(-d1)
        return p
def chooserOption(S,X,T,r,sigma,tao):
    call_T=call_or_Put (S,X,T,r,sigma,T)
    put_tao=call_or_Put (S,X,T,r,sigma,tao,type='P')
    return call_T- put_tao
```

```
In [81]: print(f"price of a chooser option={price}")
price of a chooser option=2.256
```

```
s=40.          # stock price today
x=40.          # exercise price
T=6./12        # maturity date in years
tao=1./12.     # when to choose
r=0.05         # risk-free rate
sigma=0.2      # volatility
#
price=round(chooserOption(s,x,T,r,sigma,tao),3)
print(f"price of a chooser option={price}")
```